

# Afriticket

## M-Pesa STK Push Setup Guide

Sandbox-to-production deployment checklist

---

Prepared for the Afriticket ticketing platform

Updated: 2026-09-27

# Contents

|                                                           |   |
|-----------------------------------------------------------|---|
| 1 Purpose and scope .....                                 | 1 |
| 1.1 Values used by this Afriticket build .....            | 1 |
| 2 Architecture at a glance .....                          | 1 |
| 3 Step 1 — Confirm prerequisites .....                    | 1 |
| 4 Step 2 — Apply the Afriticket database migrations ..... | 2 |
| 5 Step 3 — Create a Daraja application .....              | 3 |
| 6 Step 4 — Generate the callback secret .....             | 3 |
| 7 Step 5 — Deploy the Edge Functions .....                | 4 |
| 8 Step 6 — Set Supabase secrets .....                     | 4 |
| 9 Step 7 — Configure the frontend .....                   | 4 |
| 10 Step 8 — Register the callback in Daraja .....         | 5 |
| 11 Step 9 — Run the sandbox smoke test .....              | 5 |
| 12 Step 10 — Inspect logs and troubleshoot .....          | 6 |
| 13 Step 11 — Move from sandbox to production .....        | 7 |
| 14 Final launch checklist .....                           | 7 |
| 15 Reference links .....                                  | 8 |

# 1 Purpose and scope

This guide explains how to connect Afriticket to Safaricom M-Pesa Daraja using the repository's real STK Push adapter. It covers the database, Supabase Edge Functions, Daraja credentials, the callback URL, sandbox testing, and the production switch.

The integration uses **STK Push** (also known as Lipa na M-Pesa Online). A buyer enters a Kenyan phone number in Afriticket, the `initiate-payment` Edge Function requests an STK prompt, and Safaricom later calls `mpesa-webhook`. The webhook confirms the payment and issues tickets only after the callback is validated.

**Important money-movement boundary** Never put the Daraja consumer secret, passkey, service-role key, or webhook secret in the frontend `.env` file. Only `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` belong in the browser build.

## 1.1 Values used by this Afriticket build

- **Account reference:** Afriticket
- **Transaction description:** Payment for Afriticket
- **Transaction type:** CustomerBuyGoodsOnline
- **Business shortcode:** 3432873 as supplied for this deployment
- **PartyB:** blank, as required by the supplied buy-goods payload
- **Callback authentication:** a long random secret included as the final URL path segment

The shortcode must be the shortcode Safaricom has assigned to the Daraja environment you are using. If Safaricom gives you a different sandbox or production shortcode, use that value in the corresponding environment's Supabase secret.

## 2 Architecture at a glance

| Component                     | Responsibility                                                                                                                                             |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CheckoutDialog                | Creates server-priced holds and invokes <code>initiate-payment</code> .                                                                                    |
| <code>initiate-payment</code> | Reads the order total from Supabase and sends the STK Push through Daraja.                                                                                 |
| <code>mpesa-webhook</code>    | Receives Safaricom's callback, validates the secret path, deduplicates it, and confirms or fails the payment.                                              |
| Database RPCs                 | <code>record_payment_initiation</code> , <code>confirm_payment_and_issue_tickets</code> , and <code>fail_payment</code> maintain payment and ticket state. |

## 3 Step 1 — Confirm prerequisites

Before configuring payments, confirm the following:

- ☑ You have access to the Afriticket GitHub repository and local project directory.
- ☑ You have a Supabase project for staging. Use a separate project for production.

- ☑ The Supabase CLI is installed and authenticated.
- ☑ The frontend will be served from HTTPS in staging/production.
- ☑ You have a Safaricom Daraja account and can create an API app.
- ☑ You have decided which project will use sandbox and which will use production money movement.

From the project root, confirm the current branch and CLI:

```
git branch --show-current
supabase --version
```

**Use staging first** Do not point a real production shortcode at an untested frontend or an unverified callback. Test with Daraja sandbox first, then repeat the configuration in a separate production Supabase project.

## 4 Step 2 — Apply the Afriticket database migrations

Run all migrations in filename order. The M-Pesa functions depend on the payment tables and RPCs created earlier in the chain.

The current repository contains migrations 0001 through 0020. If using the Supabase CLI and the project is linked:

```
supabase link --project-ref <staging-project-ref>
supabase db push
```

If using the Supabase SQL Editor, paste and run each file separately in this order:

```
20260101000001_extensions_and_identity.sql
20260101000002_catalog.sql
20260101000003_inventory.sql
20260101000004_orders_and_payments.sql
20260101000005_tickets_notifications_audit.sql
20260101000006_row_level_security.sql
20260101000007_organiser_applications.sql
20260101000008_inventory_functions.sql
20260101000009_event_status_transitions.sql
20260101000010_payments_tickets_checkin.sql
20260101000011_rate_limiting_and_order_limit_fix.sql
20260101000012_events_public_view_and_org_lock.sql
20260101000013_operational_metrics.sql
20260101000014_data_export_and_deletion.sql
20260101000015_landing_page_settings.sql
20260101000016_wallpaper_storage_bucket.sql
20260101000017_multi_ticket_checkout.sql
20260101000018_notification_delivery_worker.sql
20260101000019_event_cover_images.sql
20260101000020_category_management.sql
```

Verify that the payment schema exists:

```
select to_regclass('public.orders') as orders,
       to_regclass('public.payments') as payments,
       to_regclass('public.payment_webhook_events') as webhook_events;

select routine_name
from information_schema.routines
where routine_schema = 'public'
and routine_name in (
    'record_payment_initiation',
    'confirm_payment_and_issue_tickets',
    'fail_payment'
);
```

All three tables and all three routines must be present before continuing. If `public.events` or `public.orders` is missing, stop and apply the earlier migrations first; migrations 0015–0020 are extensions, not a replacement for the base schema.

## 5 Step 3 — Create a Daraja application

1. Open the official [Daraja Developer Portal](#).
2. Create an account or sign in.
3. Create a new application for the Afriticket staging environment.
4. Enable the M-PESA Express / STK Push API for the app.
5. Copy the **Consumer Key** and **Consumer Secret** into a secure password manager.
6. Record the sandbox shortcode and passkey shown by Daraja. For this Afriticket deployment, the requested shortcode is 3432873; do not assume the generic Daraja demo shortcode is interchangeable with it.
7. Keep the sandbox base URL as `https://sandbox.safaricom.co.ke`.

The Afriticket adapter calls these Daraja endpoints:

```
GET /oauth/v1/generate?grant_type=client_credentials
POST /mpesa/stkpush/v1/processrequest
POST /mpesa/stkpushquery/v1/query
```

## 6 Step 4 — Generate the callback secret

The callback URL includes a secret path segment because Daraja callbacks are not signed by the adapter. Generate a high-entropy secret locally:

```
openssl rand -hex 32
```

Copy the output into a password manager. Do not commit it to GitHub, paste it into the frontend `.env`, or put it in a screenshot. In the examples below, replace `<callback-secret>` with that value.

The resulting callback URL will be:

```
https://<project-ref>.supabase.co/functions/v1/mpesa-webhook/<callback-secret>
```

The final path segment must be exactly the same in both `MPESA_CALLBACK_URL` and `MPESA_WEBHOOK_SECRET`.

## 7 Step 5 — Deploy the Edge Functions

From the Afriticket project root, deploy the STK initiation function and the public webhook function:

```
supabase link --project-ref <staging-project-ref>
supabase functions deploy initiate-payment
supabase functions deploy mpesa-webhook --no-verify-jwt
```

The `--no-verify-jwt` flag is required for `mpesa-webhook` because Safaricom sends the callback without a Supabase Authorization header. The function still authenticates callbacks using the secret path segment before processing the body.

The function URLs are:

`https://<project-ref>.supabase.co/functions/v1/initiate-payment`

`https://<project-ref>.supabase.co/functions/v1/mpesa-webhook/<callback-secret>`

## 8 Step 6 — Set Supabase secrets

Set the secrets in the same Supabase project where the functions were deployed. The following is the complete staging configuration for the Afriticket adapter:

```
supabase secrets set \
  PAYMENT_PROVIDER=mpesa \
  ALLOWED_ORIGIN=https://<staging-frontend-domain> \
  MPESA_CONSUMER_KEY='<daraja-consumer-key>' \
  MPESA_CONSUMER_SECRET='<daraja-consumer-secret>' \
  MPESA_SHORTCODE=3432873 \
  MPESA_PASSKEY='<daraja-passkey>' \
  MPESA_BASE_URL=https://sandbox.safaricom.co.ke \
  MPESA_CALLBACK_URL='https://<project-ref>.supabase.co/functions/v1/mpesa-webhook/<callback-secret>' \
  MPESA_WEBHOOK_SECRET='<callback-secret>'
```

`SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` are automatically injected by Supabase Edge Functions. Do not set or expose them in the frontend.

To inspect secret names without printing values:

```
supabase secrets list
```

After changing secrets, redeploy both payment functions so the deployment is unquestionably using the new values:

```
supabase functions deploy initiate-payment
supabase functions deploy mpesa-webhook --no-verify-jwt
```

## 9 Step 7 — Configure the frontend

In the frontend project's `.env`, set only the public Supabase variables:

```
VITE_SUPABASE_URL=https://<project-ref>.supabase.co
VITE_SUPABASE_ANON_KEY=<supabase-anon-public-key>
```

Restart the frontend after changing `.env` and rebuild it:

```
npm install
npm run lint
npm run build
```

Deploy the resulting frontend to the staging domain used in `ALLOWED_ORIGIN`. The browser invokes `initiate-payment` through Supabase; it does not call Safaricom directly.

## 10 Step 8 — Register the callback in Daraja

In the Daraja application settings, register the **exact** value of `MPESA_CALLBACK_URL` as the STK Push `CallbackURL`.

Check all of the following:

- ☑ The URL uses HTTPS.
- ☑ The project reference is the same Supabase project used for the deployed functions.
- ☑ The path ends with `/functions/v1/mpesa-webhook/<callback-secret>`.
- ☑ There is no trailing space, accidental quote, or missing secret segment.
- ☑ The callback URL is reachable from the public internet, not only from localhost.

The STK request generated by this Afriticket code contains:

```
{
  "BusinessShortCode": "3432873",
  "TransactionType": "CustomerBuyGoodsOnline",
  "PartyA": "<<customer-phone>",
  "PartyB": "",
  "CallbackURL": "<MPESA_CALLBACK_URL>",
  "AccountReference": "Afriticket",
  "TransactionDesc": "Payment for Afriticket"
}
```

The password is generated at request time as `Base64(shortcode + passkey + timestamp)`. Do not precompute or store it.

## 11 Step 9 — Run the sandbox smoke test

1. Sign in to Afriticket with a buyer account.
2. Confirm that the frontend is using the intended Supabase project.
3. Open a published event with an available ticket type.
4. Select a ticket and continue to M-Pesa payment.
5. Enter the Daraja sandbox test MSISDN in the format expected by the Daraja sandbox, normally `2547XXXXXXX`.
6. Submit the checkout.
7. Confirm that the order becomes `awaiting_payment` and an STK prompt is simulated by the sandbox.
8. Complete the sandbox prompt using Daraja's documented test instructions.
9. Confirm that the callback reaches `mpesa-webhook`.
10. Confirm that the order becomes paid and the ticket appears under **My tickets**.

Check the database after the test:

```
select o.reference, o.status as order_status,
       p.provider, p.provider_reference, p.status as payment_status,
       p.confirmed_at
from public.orders o
join public.payments p on p.order_id = o.id
order by p.created_at desc
limit 10;
```

A successful flow should show `order_status = 'paid'`, `payment_status = 'succeeded'`, a Daraja `provider_reference`, and a non-null `confirmed_at`.

## 12 Step 10 — Inspect logs and troubleshoot

Tail the two payment functions while testing:

```
supabase functions logs initiate-payment
supabase functions logs mpesa-webhook
```

Common failures:

| Symptom                                             | Likely cause and action                                                                                                                                                              |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| M-Pesa auth failed                                  | Consumer key/secret mismatch, wrong base URL, or sandbox app not enabled. Re-copy both credentials and keep the sandbox host.                                                        |
| STK push rejected                                   | Shortcode/passkey mismatch, invalid phone format, unsupported transaction type for the shortcode, or an amount rejected by the environment.                                          |
| Callback never arrives                              | Wrong callback URL, function deployed with JWT verification still enabled, non-HTTPS URL, or callback secret mismatch.                                                               |
| Webhook returns 401                                 | The final URL path segment does not equal <code>MPESA_WEBHOOK_SECRET</code> .                                                                                                        |
| Payment callback is accepted but order is unchanged | The callback's <code>CheckoutRequestID</code> does not match the recorded provider reference, or the database migration/RPCs are incomplete.                                         |
| Frontend says function is not deployed              | <code>initiate-payment</code> is missing from the same Supabase project used by <code>VITE_SUPABASE_URL</code> , or the frontend was built before the environment values were added. |

To check whether the public webhook route is reachable, send a harmless authenticated-shaped callback with a nonexistent checkout ID. This should not confirm a real order:

```
curl -i -X POST \
  'https://<project-ref>.supabase.co/functions/v1/mpesa-webhook/<callback-secret>' \
  -H 'content-type: application/json' \
  --data '{"Body":{"stkCallback":{"MerchantRequestID":"smoke","CheckoutRequestID":"smoke-unknown","ResultCode":1032,"ResultDesc":"Smoke test"}}}'
```

Do not use a real `CheckoutRequestID` in this smoke test.

## 13 Step 11 — Move from sandbox to production

Production is a separate configuration, not a code edit:

1. Create or obtain the production Daraja application and its approved shortcode/passkey.
2. Confirm the production shortcode supports CustomerBuyGoodsOnline for the intended merchant account.
3. Deploy a separate production Supabase project or confirm the project separation policy.
4. Deploy the same two Edge Functions to the production project.
5. Generate a new production callback secret. Never reuse the staging secret.
6. Set production secrets with `MPESA_BASE_URL=https://api.safaricom.co.ke` and the production shortcode, passkey, consumer key, consumer secret, callback URL, and webhook secret.
7. Register the production callback URL in the production Daraja application.
8. Set the production frontend's `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` to the production project.
9. Set `ALLOWED_ORIGIN` to the exact production frontend origin.
10. Perform a controlled low-value production test only after Safaricom confirms the merchant account is enabled.
11. Monitor function logs and `metric_payment_reconciliation` after launch.

**Production safety** Do not switch `PAYMENT_PROVIDER=mpesa` in a production project until the production shortcode, passkey, callback registration, refund policy, and settlement process have been confirmed. This integration records buyer-to-platform payment state; organiser settlement is a separate operational process.

## 14 Final launch checklist

- ☑ Migrations 0001–0020 are applied in order.
- ☑ ``record_payment_initiation``, ``confirm_payment_and_issue_tickets``, and ``fail_payment`` exist.
- ☑ Both Edge Functions are deployed to the intended project.
- ☑ ``mpesa-webhook`` is deployed with ``--no-verify-jwt``.
- ☑ All ``MPESA_*`` secrets are set only in Supabase Edge Function secrets.
- ☑ The shortcode is correct for the selected Daraja environment.
- ☑ The callback URL and webhook secret use the same random path segment.
- ☑ Daraja has the exact callback URL registered.
- ☑ The frontend has only public ``VITE_SUPABASE_*`` values.
- ☑ ``ALLOWED_ORIGIN`` matches the deployed frontend origin.
- ☑ A sandbox payment created a paid order and issued a ticket.
- ☑ A failed or cancelled payment released the inventory hold.
- ☑ Payment and webhook logs are visible to the operator responsible for launch.
- ☑ M-Pesa refunds and organiser settlement have an agreed manual/operational process.

## 15 Reference links

- [Safaricom Daraja Developer Portal](#)
- [Afriticket GitHub repository](#)
- Repository deployment notes: `supabase/functions/README.md` and `supabase/DEPLOYMENT.md`
- Repository payment adapter: `supabase/functions/_shared/mpesaProvider.ts`